

Module 4

Vector Search & Hybrid Search

Cursus AI Engineer · Neosoft · 2 jours

Positionnement du module

Vous savez désormais (Modules 1-3) :

- Choisir un modèle, comprendre les embeddings, chunker un corpus
- Préparer des données documentaires propres pour l'IA
- Cadrer un projet d'assistant IA

Ce module construit le **socle technique du RAG** :

- Stocker des vecteurs efficacement (Qdrant, pgvector)
- Récupérer les bons chunks à grande échelle
- Combiner dense et sparse pour maximiser la pertinence

*Sans recherche fiable, **pas de RAG fiable**. C'est ici que tout se joue.*

Objectifs pédagogiques

À l'issue des 2 jours, vous serez capable de :

1. **Déployer** une base vectorielle production-ready (Qdrant + pgvector)
2. **Indexer** un corpus avec stratégies avancées (parent-document, contextual)
3. **Concevoir** une recherche hybride dense + sparse + fusion RRF
4. **Filtrer** par métadonnées en multi-tenant
5. **Benchmark** rigoureusement : recall@k, MRR, latence p95
6. **Tuner** les paramètres HNSW pour un compromis recall/latence donné

Plan des 2 jours

Jour 1 — Recherche vectorielle dense

Bloc	Contenu	Durée
Matin	Pipeline complet de recherche vectorielle	45 min
	Métriques de distance et leur impact	30 min
	Index HNSW : théorie et tuning	1h
	Démo Qdrant + TP indexation	1h45
Après-midi	pgvector : alternative SQL	1h
	Stratégies avancées (parent-document, contextual)	1h
	TP indexation production-ready	1h30

Plan des 2 jours

Jour 2 — Recherche hybride et benchmarking

Bloc	Contenu	Durée
Matin	Limites du dense pur, intro sparse	30 min
	BM25 et SPLADE : recherche lexicale moderne	1h
	Fusion RRF : théorie et pratique	1h
	TP recherche hybride	1h
Après-midi	Multi-tenant : 3 stratégies d'isolation	1h
	Benchmarking : recall, MRR, latence	1h
	TP final : benchmark complet sur corpus réel	1h30

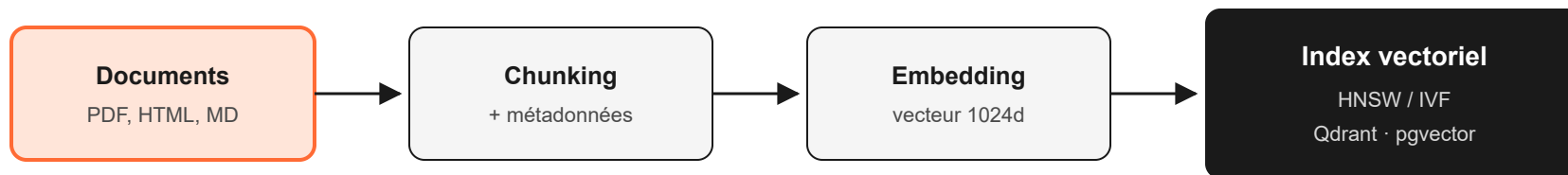
Jour 1 - Partie 1

Pipeline de recherche vectorielle

Le pipeline complet

Pipeline complet d'une recherche vectorielle

↓ Phase 1 — Indexation (offline)



↓ Phase 2 — Recherche (online, à chaque requête)



🕒 Performances cibles en production

- Embedding requête : 20-100 ms (selon modèle, local vs cloud)
- Recherche kNN sur 1M chunks (HNSW) : < 50 ms
- Re-ranking cross-encoder top-20 : 100-300 ms
- Total p95 : < 500 ms (sans LLM final)

Deux phases asymétriques

Indexation (offline, occasionnel) :

- On peut prendre du temps : qualité > vitesse
- Coût payé une fois (avec mises à jour incrémentales)
- 5-50 ms par chunk selon le modèle d'embedding

Recherche (online, à chaque requête) :

- Latence critique : SLA souvent < 500 ms
- On ne peut pas refaire d'embeddings côté chunks
- Le coût scale avec le nombre de requêtes

Règle d'or : ne jamais faire un travail à la requête qui pourrait être fait à l'indexation.

Le piège classique du débutant

✗ « J'embedding chaque chunk **et la requête** avec le même modèle, ça va marcher »

Pas si simple. Plusieurs subtilités :

1. **Préfixes** : certains modèles (e5, bge) attendent `query:` vs `passage:`
2. **Normalisation** : si le modèle est entraîné sur vecteurs normalisés, il faut les normaliser
3. **Tokenisation** : la longueur max d'un embedding (512-8192 tokens) doit matcher la stratégie de chunking
4. **Multilingue** : un modèle anglais sur du français = catastrophe

*Lisez **toujours** la model card avant d'utiliser un modèle d'embedding en production.*

Le rôle des métadonnées dans le pipeline

Chaque chunk porte ses métadonnées :

```
{
  "id": "doc_42_chunk_3",
  "vector": [0.23, -0.11, ...],      # 1024 dims
  "payload": {                      # métadonnées
    "content": "Pour configurer SAML...",
    "source": "guide_admin.pdf",
    "page": 12,
    "product": "NovaCloud",
    "version": "3.2",
    "language": "fr",
    "indexed_at": "2026-05-07"
  }
}
```

À la requête : `filter = {"product": "NovaCloud", "version": ">=3.0"}` **avant** la recherche kNN → moins de candidats à scanner, recall sur le sous-ensemble pertinent.

Jour 1 - Partie 2

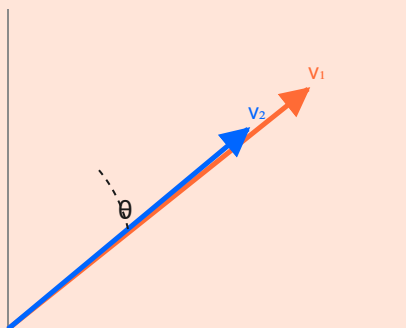
Métriques de distance

Trois métriques à connaître

Métriques de distance — laquelle choisir ?

Cosine

mesure l'angle



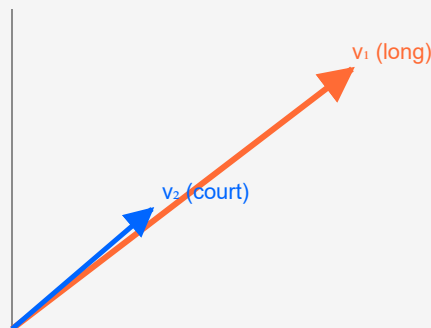
$$\cos(\theta) \in [-1, 1]$$

- ✓ Insensible à la magnitude
- ✓ Standard pour le texte
- ✓ Identique à dot si vecteurs normalisés

→ **Choix par défaut pour RAG**

Dot product

produit scalaire

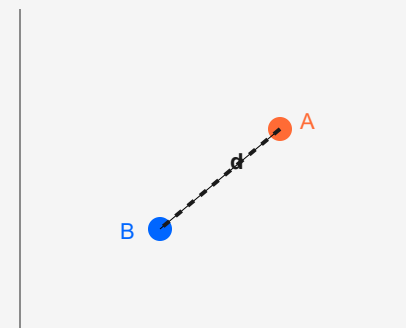


$$\mathbf{v}_1 \cdot \mathbf{v}_2 = \sum v_i \times w_i$$

- ✓ Plus rapide à calculer
 - ✓ Sensible à la magnitude
 - ⚠ Peut biaiser vers les vecteurs longs
- **Si modèle entraîné dot (e5, OpenAI)**

Euclidean (L2)

distance physique



$$d = \sqrt{\sum (v_i - w_i)^2}$$

- ✓ Intuitive (distance "physique")
 - ⚠ Sensible à la magnitude
 - ⚠ Peu utilisée pour le texte
- **Plutôt pour images / audio**

Comment choisir la métrique

Règle absolue : utilisez la métrique sur laquelle votre modèle a été **entraîné**.

Modèle	Métrique recommandée
text-embedding-3-* (OpenAI)	cosine ou dot
bge-m3 , bge-large-*	cosine
multilingual-e5-*	cosine (vecteurs normalisés)
voyage-3	cosine
mistral-embed	cosine

*Si vous normalisez les vecteurs ($||v|| = 1$), **dot et cosine sont équivalents**.*

Pourquoi rarement euclidien pour le texte

L'embedding d'un texte long a une **magnitude plus grande** que celui d'un texte court.

→ La distance euclidienne fait apparaître les textes courts comme « plus proches ».

→ Mauvais signal : un chunk long peut être très pertinent.

Cosine ignore la magnitude et compare uniquement la direction (= le sens). C'est le bon comportement pour le texte.

*Euclidien reste utile pour des embeddings de **vision** ou **audio** où la magnitude porte du sens.*

Jour 1 - Partie 3

Index HNSW

Le problème : le k-NN brute force ne scale pas

Sur 1 million de chunks en 1024 dimensions :

- **Brute force** : $1\text{ M} \times 1024$ multiplications + tri = **~500 ms par requête**
- **Avec 10 000 requêtes/seconde** : impossible

Solution : structures d'index approximatives (ANN — Approximate Nearest Neighbor).

Les principales :

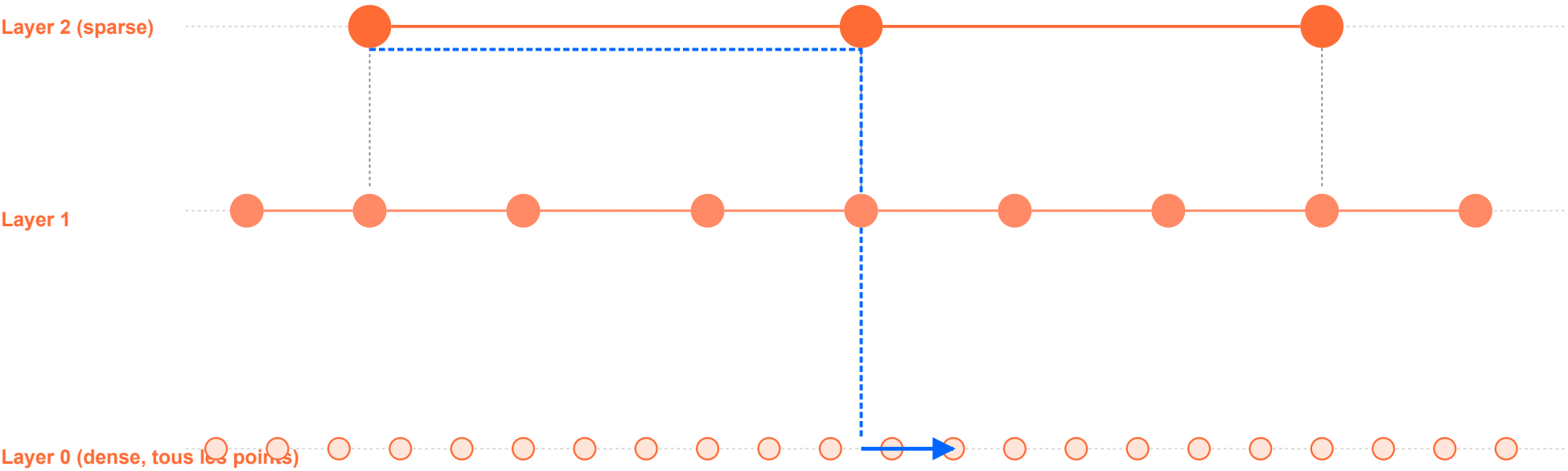
- **HNSW** : Hierarchical Navigable Small World — utilisé partout
- **IVF** : Inverted File Index — bon avec PQ pour très gros volumes
- **DiskANN** : pour milliards de vecteurs sur disque

*Compromis : on accepte un **recall** < 100% pour gagner 1000× en latence.*

HNSW expliqué

HNSW — Hierarchical Navigable Small World

Index multi-couches : recherche descendante du plus grossier au plus fin



Paramètres clés à connaître

M : nb max de connexions par nœud (16-64). Plus haut = qualité ↑, mémoire ↑

ef_construction : qualité du graphe à l'indexation (100-500). Plus haut = build lent mais meilleur recall

ef_search : largeur de recherche (réglable à la requête)

ef_search : largeur de recherche (réglable à la requête). Plus haut = meilleur recall vs latence. Tuning empirique sur votre golden set

HNSW — comment ça marche

1. **À l'insertion** : chaque vecteur est ajouté à un niveau aléatoire (loi exponentielle)
2. **Layer haut** : peu de nœuds, longues connexions → navigation rapide à grande échelle
3. **Layer bas** : tous les nœuds, connexions courtes → recherche fine
4. **À la requête** : on entre par le top, on descend en suivant le voisin le plus proche

Complexité : $O(\log N)$ au lieu de $O(N)$.

Sur 1M chunks : ~10-50 ms au lieu de 500 ms. Avec un recall@10 typique de 95-99%.

Tuner HNSW : 3 paramètres

Paramètre	Effet	Valeurs typiques
M	Nb max de connexions par nœud	16-64
ef_construction	Qualité du graphe à l'indexation	100-500
ef_search	Largeur de recherche à la requête	50-200

Compromis :

- ↑ **M** : meilleur recall, plus de mémoire
- ↑ **ef_construction** : meilleur index, plus long à construire
- ↑ **ef_search** : meilleur recall, latence plus élevée (réglable à la requête)

✓ *Tuner **ef_search** dynamiquement selon le SLA voulu, c'est gratuit.*

Quand préférer IVF à HNSW

Critère	HNSW gagne	IVF gagne
Volume	< 10M vecteurs	> 50M vecteurs
RAM disponible	Vous en avez	Limitée
Latence < 50ms	✓	Plus difficile
Mises à jour fréquentes	✓	Reconstruction nécessaire
Compression PQ	Limité	✓ Très efficace

En 2026, **HNSW est le défaut** sauf cas extrêmes. Tous les vector DB modernes l'implémentent.

Jour 1 - Partie 4

Qdrant en pratique

Pourquoi Qdrant

Open-source, écrit en Rust, conçu spécifiquement pour la recherche vectorielle.

Forces :

- Filtres complexes (AND/OR/NOT) **avant** kNN — pas de pré-filtrage faux positif
- Payload riche (JSON arbitraire par point)
- Sharding et réplication natifs
- Quantization (scalar, product, binary)
- Hybrid search (dense + sparse) intégré

Alternatives : Weaviate, Milvus, Pinecone (SaaS), Vespa, LanceDB.

*Pour ce cours : **Qdrant** comme référence + **pgvector** comme alternative SQL.*

Démarrer Qdrant en 30 secondes

```
docker run -p 6333:6333 -p 6334:6334 \
-v $(pwd)/qdrant_storage:/qdrant/storage \
qdrant/qdrant:latest
```

UI Web : <http://localhost:6333/dashboard>

```
from qdrant_client import QdrantClient
client = QdrantClient(url="http://localhost:6333")
client.get_collections() #  ça marche
```

En production : `qdrant/qdrant-cLOUD` (managé) ou cluster K8s avec opérateur officiel.

Créer une collection

```
from qdrant_client import QdrantClient
from qdrant_client.models import VectorParams, Distance, PayloadSchemaType

client = QdrantClient(url="http://localhost:6333")

client.create_collection(
    collection_name="docs",
    vectors_config=VectorParams(size=1024, distance=Distance.COSINE),
)

# Index sur les champs filtrables → 100x plus rapide en filtrage
client.create_payload_index("docs", "product", PayloadSchemaType.KEYWORD)
client.create_payload_index("docs", "version", PayloadSchemaType.KEYWORD)
client.create_payload_index("docs", "indexed_at", PayloadSchemaType.DATETIME)
```

⚠ *Les **payload indexes** sont ESSENTIELS dès que vous filtrez. Sans eux : scan complet du payload.*

Indexer des points

```
from qdrant_client.models import PointStruct

points = [
    PointStruct(
        id=i,
        vector=embeddings[i].tolist(),
        payload={
            "content": chunks[i].text,
            "source": chunks[i].source,
            "product": "NovaCloud",
            "version": "3.2",
        }
    )
    for i in range(len(chunks))
]

# Upsert par batches de 100-500 pour la performance
client.upsert(collection_name="docs", points=points, wait=True)
```

Indexer 100k chunks ≈ 1-3 minutes. La majorité du temps = embeddings, pas l'index.

Recherche avec filtre

```
from qdrant_client.models import Filter, FieldCondition, MatchValue

response = client.query_points(
    collection_name="docs",
    query=query_embedding,
    query_filter=Filter(
        must=[
            FieldCondition(key="product", match=MatchValue(value="NovaCloud")),
            FieldCondition(key="version", match=MatchValue(value="3.2")),
        ]
    ),
    limit=10,
    with_payload=True,
)

for hit in response.points:
    print(f"{hit.score:.3f} | {hit.payload['source']} | {hit.payload['content'][:80]}")
```

Quantization : compresser sans perdre

Pour économiser de la RAM (3-32x selon le mode) :

```
from qdrant_client.models import ScalarQuantization, ScalarQuantizationConfig, ScalarType

client.create_collection(
    "docs",
    vectors_config=VectorParams(size=1024, distance=Distance.COSINE),
    quantization_config=ScalarQuantization(
        scalar=ScalarQuantizationConfig(
            type=ScalarType.INT8,
            always_ram=True, # vectors quantisés en RAM, originaux sur disque
            quantile=0.99,
        )
    ),
)
```

Mode	Compression	Perte recall typique
Scalar (int8)	4x	~1%
Product (PQ)	8-32x	2-5%
Binary	32x	5-15% (mais ultra rapide)

Jour 1 - Partie 5

pgvector — l'alternative SQL

pgvector : quand le choisir

pgvector = extension PostgreSQL qui ajoute un type `vector` et des index ANN.

Choisir pgvector si :

-  Vous avez déjà PostgreSQL en production
-  Volume modéré (< 5M vecteurs)
-  Vous voulez des **JOINS SQL** entre vecteurs et données métier
-  Transactions ACID requises (cas réglementé)
-  Une seule équipe à former (DBAs SQL existants)

Choisir Qdrant si :

-  Volume > 10M vecteurs
-  Filtres très complexes ou changements fréquents de schéma
-  SLA latence < 50 ms en p95

pgvector — installation et création

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE chunks (
  id BIGSERIAL PRIMARY KEY,
  content TEXT NOT NULL,
  embedding vector(1024) NOT NULL,
  source TEXT,
  product TEXT,
  version TEXT,
  indexed_at TIMESTAMPTZ DEFAULT NOW()
);

-- Index HNSW (PostgreSQL 16+ avec pgvector 0.5+)
CREATE INDEX ON chunks
USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Index B-tree classique pour les filtres
CREATE INDEX ON chunks (product, version);
```

pgvector — requêtes

```
-- Recherche kNN simple
SELECT id, content, source,
       1 - (embedding <=> '[0.1, 0.2, ...]')::vector) AS similarity
FROM chunks
ORDER BY embedding <=> '[0.1, 0.2, ...]')::vector
LIMIT 10;

-- Avec filtre métier
SELECT id, content,
       1 - (embedding <=> $1) AS similarity
FROM chunks
WHERE product = 'NovaCloud' AND version >= '3.0'
ORDER BY embedding <=> $1
LIMIT 10;

-- ⚠ ef_search réglable par session
SET hnsw.ef_search = 100;
```

Les opérateurs : `<->` L2, `<=>` cosine, `<#>` dot product (négatif).

Le piège du filtre + ANN dans pgvector

⚠ **Filtre WHERE + ORDER BY embedding** : PostgreSQL peut :

1. **Plan A** : utiliser l'index HNSW puis filtrer (rapide, mais peut renvoyer < 10 résultats)
2. **Plan B** : utiliser l'index B-tree pour filtrer puis scan séquentiel (lent si beaucoup de résultats)

Solution : `ef_search` plus grand pour compenser la perte.

```
-- Si beaucoup de résultats sont filtrés, augmenter ef_search  
SET hnsw.ef_search = 200; -- au lieu de 40 par défaut
```

Sur Qdrant : pas ce problème car le filtrage est pendant la traversée du graphe.

Jour 1 - Partie 6

Stratégies avancées

Parent-document retrieval

Problème : un chunk court (200 tokens) embedde bien sémantiquement, mais le LLM a besoin d'un contexte plus large pour répondre.

Solution : indexer des **petits chunks**, mais retourner les **gros parents** au LLM.

```
Document
├─ Section A
│   ├── Chunk A1 (200 tok) ← indexé
│   ├── Chunk A2 (200 tok) ← indexé
│   └── Chunk A3 (200 tok) ← indexé
└─ Section B (parent ← retourné si A1, A2 ou A3 matchent)
```

Avantages : précision du retrieval ↑, contexte du LLM ↑, déduplication automatique.

Implémentation parent-document

```
# À l'indexation : chaque petit chunk garde une référence à son parent
points = [
    PointStruct(
        id=f"{parent_id}_chunk_{i}",
        vector=embed(small_chunk),
        payload={
            "content": small_chunk,
            "parent_id": parent_id,
            "parent_content": large_parent_chunk, # le gros chunk
        }
    )
    for parent_id, parent_content in parents.items()
    for i, small_chunk in enumerate(split_small(parent_content))
]

# À la requête : récupérer top-k petits chunks, déduplicates par parent_id
hits = client.query_points(...).points
unique_parents = {}
for hit in hits:
    pid = hit.payload["parent_id"]
    if pid not in unique_parents:
        unique_parents[pid] = hit.payload["parent_content"]
```

Contextual retrieval (Anthropic, 2024)

Idée : enrichir chaque chunk avec un **contexte synthétique** avant indexation.

```
def add_context(chunk: str, full_doc: str) -> str:
    prompt = f"""<document>{full_doc}</document>
<chunk>{chunk}</chunk>
Donne en 50 tokens max le contexte de ce chunk dans le document.
"""
    context = llm(prompt, max_tokens=80)
    return f"{context}\n\n{chunk}"

# Indexer le chunk enrichi
embedding = embed(add_context(chunk, full_doc))
```

Anthropic a montré -49% d'erreurs retrieval avec cette technique. Coût : un appel LLM par chunk à l'indexation. Économisé via prompt caching du document complet.

Hypothetical Document Embeddings (HyDE)

Idée : générer une **réponse hypothétique** à la requête, et embedder cette réponse plutôt que la requête elle-même.

```
# Au lieu de embed("Comment configurer SAML ?")
hypothetical = llm("Réponds brièvement : Comment configurer SAML ?")
# → "Pour configurer SAML 2.0, allez dans Paramètres > Auth..."

results = search(embed(hypothetical))
```

Pourquoi ça marche : la réponse hypothétique a une distribution sémantique plus proche des **documents réels** que la question.

Coût : 1 appel LLM en + par requête. À mesurer si le gain compense.

Jour 1 - Partie 7

TPs de la journée

TP 1.1 — Premier index Qdrant

Objectif : indexer 1000 articles Wikipédia français et faire 5 requêtes types.

→ Voir `exercises/exo1_qdrant_basics.md`

TP 1.2 — Indexation production-ready

Objectif : ré-indexer le même corpus avec :

- Parent-document retrieval
- Index payload sur 3 champs filtrables
- Quantization int8
- Mesure : recall@10 avant/après

→ Voir `exercices/exo2_advanced_indexing.md`

Jour 2 - Partie 1

Limites du dense pur

Le mythe « les embeddings comprennent tout »

Cas réel : recherche « erreur 503 endpoint /api/reports v3.2 »

Avec recherche dense uniquement :

-  Trouve les docs sur les erreurs API en général
-  Manque le doc qui parle exactement de 503 sur /api/reports
-  Pas de match sur v3.2 (les chiffres sont mal embeddés)

Pourquoi : les embeddings encodent du **sens flou**, pas des **identifiants exacts**.

Pour les requêtes contenant des références techniques (codes, IDs, versions, noms), le dense pur perd.

Dense vs sparse — les deux mondes

Recherche dense vs sparse — forces complémentaires

Requête : « Comment configurer SAML 2.0 sur la version 3.2 ? »

Dense (vectoriel)

comprend le sens, paraphrase

Comment ça marche

- embedding 1024 dimensions
- similarité cosinus dans l'espace

Trouve bien

- ✓ « Procédure d'authentification fédérée »
- ✓ « Mise en place d'un fournisseur d'identité »
- ✓ Concepts proches mais formulés différemment

Trouve mal

- X « SAML 2.0 » exact si rare dans le corpus
- X « Version 3.2 » — les chiffres sont mal embeddés
- X Codes d'erreur précis, références techniques

Modèles : bge-m3, e5-large, OpenAI embeddings

Sparse (lexical)

match exact des termes (BM25)

Comment ça marche

- vecteur creux (vocabulaire complet)
- score TF-IDF / BM25 par token partagé

Trouve bien

- ✓ « SAML 2.0 » trouvé exactement
- ✓ « version 3.2 », noms produits, codes erreur
- ✓ Acronymes, références exactes

Trouve mal

- X Synonymes (« fédéré » ≠ « SAML »)
- X Reformulations utilisateur
- X Concepts implicites

Implémentation : BM25, Elasticsearch, Tantivy, SPLADE

Quand le sparse seul suffit

Cas où la recherche **lexicale BM25** est suffisante, voire meilleure :

- 🔍 Recherche de **codes d'erreur** dans une base de support
- 🔍 Recherche de **références produit** (« ABC-12345 »)
- 🔍 Recherche **juridique** (citations exactes, jurisprudence)
- 🔍 Recherche dans **logs** ou **métriques**
- 🔍 Documentation très **technique** avec terminologie stable

Pour le reste (questions naturelles, paraphrases, sémantique) : dense gagne.

*En pratique : 80% des cas RAG métier bénéficient de **l'hybride**.*

Jour 2 - Partie 2

BM25 et SPLADE

BM25 en 1 minute

Best Matching 25 : évolution de TF-IDF, avec saturation et normalisation par longueur.

$$\text{score}(\text{doc}, \text{query}) = \sum \text{IDF}(t) \times (\text{TF}(t, \text{doc}) \times (k1+1)) / (\text{TF}(t, \text{doc}) + k1 \times (1 - b + b \times |\text{doc}| / \text{avg_dl}))$$

Trois intuitions à retenir :

1. **TF saturé** : 10 occurrences $\neq$ 10× plus pertinent qu'1 occurrence
2. **IDF** : un terme rare apporte plus de signal qu'un terme fréquent
3. **Normalisation par longueur** : un long doc qui contient le terme n'est pas favorisé

Paramètres : $k1=1.2$, $b=0.75$ par défaut. Rarement à tuner.

Implémenter BM25

Option 1 — bibliothèque Python pure (rapide à mettre en place) :

```
from rank_bm25 import BM25Okapi

tokenized_docs = [doc.lower().split() for doc in docs]
bm25 = BM25Okapi(tokenized_docs)
scores = bm25.get_scores(query.lower().split())
top_k_idx = scores.argsort()[-10:][::-1]
```

Option 2 — Qdrant sparse vectors (production) :

```
from qdrant_client.models import SparseVectorParams, VectorParams

client.create_collection(
    "docs",
    vectors_config={"dense": VectorParams(size=1024, distance=Distance.COSINE)},
    sparse_vectors_config={"sparse": SparseVectorParams()},
)
```

Option 3 — Elasticsearch / OpenSearch (BM25 natif optimisé).

SPLADE — le sparse moderne

SPLADE : sparse model **appris** par un transformer (au lieu d'être basé sur des heuristiques BM25).

- Token-level expansion : « voiture » génère aussi des poids sur « véhicule », « auto », « véhicule motorisé »
- Compatibles avec les index sparse classiques
- Souvent meilleure que BM25, parfois proche du dense

Modèles à connaître :

- `naver/splade-v3`
- `prithivida/Splade_PP_en_v1`

Production : SPLADE est plus coûteux en CPU à l'indexation que BM25, mais 10-15% de gain en recall typique.

Jour 2 - Partie 3

Fusion RRF

Le problème de la fusion

Vous avez deux rankings :

- Dense : [doc-A, doc-C, doc-B, doc-E, doc-D]
- Sparse : [doc-F, doc-A, doc-G, doc-B, doc-H]

Comment combiner ?

- ✗ Moyenner les scores : pas comparables (cosine $\in [-1,1]$, BM25 non borné)
- ✗ Sommer les rangs : trop simpliste, doc absent d'un ranking pénalisé à mort
- ✓ **Reciprocal Rank Fusion (RRF)** : robuste, paramétré par `k`

Reciprocal Rank Fusion expliqué

Reciprocal Rank Fusion (RRF) — combiner dense et sparse

Ranking dense

1. doc-A « SAML guide général »

2. doc-C « Auth fédérée v3 »

3. doc-B « SSO entreprise »

4. doc-E « OIDC migration »

5. doc-D « LDAP setup »

Formule RRF

$$\text{score}(d) = \sum 1 / (k + \text{rank}_i(d))$$

$k = 60$ (constante par défaut)
somme sur chaque ranker (dense, sparse)

Ranking sparse (BM25)

1. doc-F « SAML 2.0 v3.2 spec »

2. doc-A « SAML guide général »

3. doc-G « Configuration v3.2 »

4. doc-B « SSO entreprise »

5. doc-H « LDAP v3 »

→ Ranking final fusionné

1. doc-A (dans les 2) · 2. doc-F (sparse exact) · 3. doc-B (dans les 2) · 4. doc-C (dense) · 5. doc-G (sparse)

Implémenter RRF

```
def rrf_fusion(rankings: list[list[str]], k: int = 60, top_n: int = 10) -> list[tuple[str, float]]:
    """Fusionne plusieurs rankings via Reciprocal Rank Fusion."""
    scores: dict[str, float] = {}
    for ranking in rankings:
        for rank, doc_id in enumerate(ranking, start=1):
            scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (k + rank)

    return sorted(scores.items(), key=lambda x: -x[1])[:top_n]

# Usage
dense_top20 = [hit.id for hit in dense_search(query, limit=20)]
sparse_top20 = [hit.id for hit in sparse_search(query, limit=20)]
fused = rrf_fusion([dense_top20, sparse_top20], k=60, top_n=10)
```

k = 60 est la valeur par défaut de la littérature (Cormack et al. 2009). Rarement à changer.

RRF côté Qdrant — natif

Depuis Qdrant 1.10, fusion native dans la base :

```
from qdrant_client.models import Prefetch, FusionQuery, Fusion

results = client.query_points(
    collection_name="docs",
    prefetch=[
        Prefetch(query=dense_query, using="dense", limit=20),
        Prefetch(query=sparse_query, using="sparse", limit=20),
    ],
    query=FusionQuery(fusion=Fusion.RRF),
    limit=10,
)
```

✓ Une seule requête réseau, exécution parallèle côté serveur.

Au-delà de RRF : les alternatives

Technique	Quand l'utiliser
RRF	Défaut, robuste, sans tuning
Weighted RRF	Si vous avez un golden set pour pondérer dense vs sparse
Rank Distribution	Voitures "best of both" : $\text{score} = \alpha \times \text{score_dense} + (1-\alpha) \times \text{score_sparse_normalisé}$
Cross-encoder rerank	Top-50 fusionné, puis re-ranking précis avec un cross-encoder

Recommandation : **RRF** en première intention, + **cross-encoder** si vous voulez aller plus loin (M5).

Jour 2 - Partie 4

Multi-tenant

Pourquoi c'est critique

Beaucoup de cas d'usage entreprise sont **multi-tenant** :

- Plateforme SaaS : chaque client a ses propres documents
- Mutuelle : un assistant par adhérent
- Cabinet d'avocats : un dossier par client
- ERP : chaque entité juridique a son périmètre

Erreur fatale : un utilisateur du tenant A reçoit un chunk du tenant B. Fuite de données, RGPD, fin de contrat.

Trois stratégies d'isolation

Multi-tenant : 3 stratégies d'isolation des données

1. Collection partagée + filter

tenant_id en métadonnée

Collection unique « docs »

- chunk · tenant_A
- chunk · tenant_B
- chunk · tenant_A
- chunk · tenant_C
- chunk · tenant_B

- ✓ Index unique, économique
- ✓ Filtrage à la requête
- ⚠ Risque de fuite si filtre oublié
- ⚠ Pas d'isolation à la suppression

→ < 100 tenants, données peu sensibles

2. Une collection / tenant

isolation au niveau collection

docs_A

tenant A

docs_B

tenant B

docs_C

tenant C

- ✓ Isolation forte garantie
- ✓ Suppression atomique par tenant
- ✓ Permissions par collection
- ⚠ Coût mémoire fixe par index
- ⚠ Mauvais scaling au-delà de ~1000 tenants

→ Cible < 500 tenants, données sensibles

3. Cluster dédié

isolation infrastructure complète

Cluster Qdrant tenant A

VPC dédié · chiffrement client

Cluster Qdrant tenant B

VPC dédié · chiffrement client

- ✓ Isolation maximale (réglementaire)
- ✓ Configuration / dimensionnement par tenant
- ✓ SLA différenciés possibles

⚠ Très coûteux opérationnellement

⚠ Provisioning et patching à automatiser

→ Banques, santé, défense, multi-régions

Quelle stratégie choisir

Stratégie	Tenants	Sensibilité	Coût
Filter <code>tenant_id</code>	< 100	Faible	\$
Collection / tenant	100-500	Moyenne à élevée	\$\$
Cluster dédié	< 50	Très élevée (banque, santé)	\$\$\$\$

Pattern hybride courant : filter `tenant_id` + une couche d'**autorisation côté API** qui rejette toute requête sans `tenant_id` validé par JWT.

⚠ Ne JAMAIS se reposer uniquement sur le frontend pour passer le `tenant_id` . Validation côté backend obligatoire.

Filtre tenant en pratique

```
# Layer 1 : middleware FastAPI valide le JWT et extrait tenant_id
@app.post("/search")
def search(query: SearchRequest, auth=Depends(verify_jwt)):
    tenant_id = auth.tenant_id # ← source de vérité = JWT, pas le payload

    # Layer 2 : filtre côté Qdrant
    response = client.query_points(
        collection_name="docs",
        query=embed(query.text),
        query_filter=Filter(
            must=[FieldCondition(key="tenant_id", match=MatchValue(value=tenant_id))]
        ),
        limit=10,
    )
    return response.points

# Layer 3 : tests automatisés qui vérifient l'isolation
def test_no_cross_tenant_leak():
    # Indexer un chunk du tenant A
    # Faire une recherche avec un JWT tenant B
    # Vérifier qu'on ne voit RIEN du tenant A
    ...
```

Jour 2 - Partie 5

Benchmarking

Mesurer la qualité d'un retrieval

Trois métriques essentielles :

Métrique	Question répondue
Recall@k	Le bon doc est-il dans le top-k ?
MRR	À quel rang est le bon doc en moyenne ?
NDCG@k	Le ranking est-il bon, pas juste la présence ?

Et côté opérationnel :

- **Latence p50, p95, p99**
- **Throughput** (QPS soutenu sans dégrader p95)

Recall@k — la métrique qu'on regarde en premier

```
def recall_at_k(retrieved: list[str], relevant: set[str], k: int) -> float:
    return len(set(retrieved[:k]) & relevant) / len(relevant) if relevant else 0.0
```

Interprétation :

- `recall@5 = 1.0` : tous les docs pertinents sont dans le top-5 ✓
- `recall@5 = 0.6` : 60% des docs pertinents sont dans le top-5

Cibles typiques pour un RAG production :

- $\text{recall}@5 \geq 0.85$
- $\text{recall}@10 \geq 0.95$

En dessous, votre RAG hallucinera : il n'aura pas vu le bon contexte.

MRR — la position du premier bon résultat

```
def mrr(rankings: list[list[str]], relevants: list[set[str]]) -> float:
    total = 0.0
    for ranking, rel in zip(rankings, relevants):
        for rank, doc in enumerate(ranking, start=1):
            if doc in rel:
                total += 1.0 / rank
                break
    return total / len(rankings)
```

Interprétation :

- MRR = 1.0 : le bon doc est toujours en 1ère position
- MRR = 0.5 : le bon doc est en moyenne en 2ème position (1/2)
- MRR = 0.25 : moyenne 4ème position (1/4)

*MRR est sensible au **ranking**, recall ne l'est pas. Mesurer les deux.*

Construire un golden set

Sans golden set, vous ne pouvez pas benchmarker.

Méthodes pour le construire :

1. **Annotation humaine** (gold standard) : 100-500 paires (requête, docs pertinents)
2. **À partir de FAQ existante** : la question = requête, la réponse cite ses docs
3. **Synthétique LLM** : générer des questions à partir de chunks (puis valider humainement un sous-ensemble)
4. **Logs de production** : utiliser les vraies requêtes utilisateurs avec annotation post-hoc

Mix recommandé : 50 cas annotés humains + 200 synthétiques validés. Versioner le set.

Mesurer la latence — sans biais

```
import time, statistics

def benchmark_latency(search_fn, queries: list[str], n_warmup: int = 10) -> dict:
    # Échauffer (cache, JIT, connexion)
    for q in queries[:n_warmup]:
        search_fn(q)

    latencies = []
    for q in queries:
        t0 = time.perf_counter()
        search_fn(q)
        latencies.append((time.perf_counter() - t0) * 1000)

    return {
        "p50": statistics.median(latencies),
        "p95": sorted(latencies)[int(0.95 * len(latencies))],
        "p99": sorted(latencies)[int(0.99 * len(latencies))],
        "mean": statistics.mean(latencies),
    }
```

*Toujours **warmup**, toujours **percentiles**, jamais juste la moyenne.*

Jour 2 - Partie 6

TP final

TP final — Benchmark complet

Objectif : produire un rapport comparant 3 modes (dense, sparse, hybride) sur un corpus réel.

Étapes :

1. Indexer le corpus en dense + sparse
2. Construire un golden set de 50 questions
3. Mesurer recall@5, recall@10, MRR, latence p95 pour chaque mode
4. Produire un tableau et un graphique
5. Recommandation pour la production

→ Voir `exercises/exo3_benchmark_hybride.md`

Récapitulatif

Vous savez maintenant :

- ✓ Architecturer un pipeline de recherche vectorielle production-ready
- ✓ Choisir entre Qdrant et pgvector selon le contexte
- ✓ Tuner HNSW (M, ef_construction, ef_search) pour votre SLA
- ✓ Combiner dense et sparse via RRF
- ✓ Isoler les données en multi-tenant
- ✓ Benchmarker rigoureusement votre retrieval

Prochain module (M5) : RAG avancé et évaluation Ragas.

Questions ?

Merci pour votre attention

Support, démos et exercices : `supports/M04_vector_search/`